

Сборник заданий для практических занятий

по курсу

«Проектирование архитектуры мобильных и веб-приложений»

Содержание

1	Общие сведения	3
2	Практические задания (осенний семестр)	3
2.1	Задание № 1. Простейший REST API-сервис	3
2.2	Задание № 2. Простейший проект с использованием React	4
2.3	Задание № 3. Завершение клиентской части	4
2.4	Задание № 4. Аутентификация и авторизация	5
2.5	Задание № 5. Микросервисная архитектура	5
2.6	Задание № 6. Автоматическое тестирование микросервисов	5
2.7	Задание № 7. Автоматическое UI-тестирование	5
2.8	Экзамен	5
3	Практические задания (весенний семестр)	5
3.1	Задание № 1. Kafka и обработка событий	5
3.2	Задание № 2. Event Sourcing	6
3.3	Задание № 3. Saga Choreography	6
3.4	Задание № 4. Outbox pattern, Schema Registry и DLQ	6
3.5	Зачёт	6

1 Общие сведения

Сборник содержит задания для практических занятий по курсу «Проектирование архитектуры мобильных и веб-приложений».

Лекции проходят в форме мастер-классов, соответственно, практические занятия состоят в реализации заданий, которые по структуре повторяют работы, разобранные на мастер-классах.

2 Практические задания (осенний семестр)

2.1 Задание № 1. Простейший REST API-сервис

Реализуйте с использованием FastAPI REST API-сервис, предоставляющий возможность добавлять, удалять, изменять и получать все объекты согласно варианту, а также реализуйте возможность получения части объектов для поэтапной загрузки.

Данные должны храниться в SQLite (или в СУБД по вашему выбору). Обращение к данным должно осуществляться посредством ORM-библиотеки. Осуществляйте полную валидацию корректности передаваемых данных.

1. База данных студентов группы. Поля: фамилия, имя, отчество, пол, возраст.
2. База данных расходов семьи. Поля: товар, стоимость, количество, дата.
3. База данных загрузки аудиторий. Поля: дата и время начала, дата и время окончания, аудитория, преподаватель.
4. База данных учета доходов и расходов предпринимателя. Поля: дата, тип операции (доход/расход), объем операции, описание, корреспондент.
5. База данных велоклуба. Поля: ФИО, тип велосипеда (МТВ и др.), стаж участия в велоклубе.
6. База данных рейсов авиакомпании. Поля: дата и время вылета, аэропорт вылета, аэропорт прилета, дата и время прилета, марка самолета.
7. База данных автобусных маршрутов. Поля: номер маршрута, номер парка, время начала и окончания движения, длина маршрута в км.
8. База данных электричек. Поля: вокзал, номер поезда, количество вагонов, тип (экспресс/обычный/спутник).
9. База данных товаров интернет-магазина. Поля: название товара, категория, цена товара, описание товара.
10. База заказов интернет-магазина. Поля: ФИО заказчика, стоимость заказа, скидка (в процентах), адрес доставки.
11. База данных выборов. Поля: участок, кандидат, количество голосов.
12. База данных практических работ. Поля: практическая работа, студент, номер варианта, номер уровня, дата сдачи, оценка.
13. База данных операторов и телеканалов. Поля: название, тип (спутник, кабель, интернет), охват (количество миллионов домохозяйств), минимальная стоимость подписки.

14. База данных тарифных планов оператора. Поля: название, тип вещания (обычный/HD), флаг общедоступности.
15. База данных незаконно огороженных берегов. Поля: водный объект, регион, GPS-координаты, длина недоступного участка берега, дата фиксации нарушения.
16. База данных временного прекращения движения в метро. Поля: дата и время начала прекращения движения, дата и время окончания прекращения движения, станция начала, станция окончания.
17. База данных проката фильмов. Поля: дата, время, кинотеатр, фильм, номер зала, тип сеанса (3D/IMAX/обычный).
18. База данных эвакуированных автомобилей. Поля: улица, автостоянка, GPS-координаты, тип нарушения (стоянка на проезжей части в месте запрета, стоянка на тротуаре, стоянка на газоне), номер автомобиля, тип автомобиля (легковой/грузовой малой тоннажности/грузовой большой тоннажности).
19. База данных средних специальных учебных учреждений. Поля: название, адрес, тип подчинения (федеральный/региональный), год основания, номер лицензии, номер аккредитации, дата окончания действия аккредитации.
20. База данных поселков. Поля: название, девелопер, площадь, количество жителей.
21. База данных сухопутной военной техники. Поля: название, модель, разработчик, предприятие, стоимость, тип.
22. База данных деревьев в городе. Поля: GPS-координаты, вид дерева, округ, год посадки.
23. База данных футбольных матчей. Поля: дата, команда, команда, счет, место проведения.
24. База данных обращений жителей. Поля: дата, время, объект, заявитель, содержание обращения (до 255 символов), дата ответа, ответ на обращение (до 255 символов).
25. База данных студентов колледжа. Поля: ФИО, группа, признак бюджетности, стипендия (нет/обычная/повышенная), флаг наличия социальной стипендии, дата рождения.

2.2 Задание № 2. Простейший проект с использованием React

Реализуйте клиент с использованием React для отображения и удаления данных из сервиса, созданного в предыдущем задании.

Стек: React, Vite, npm, TypeScript, Axios, Material UI.

2.3 Задание № 3. Завершение клиентской части

Реализуйте в клиенте с использованием React:

- добавление объекта;
- изменение объекта;
- бесконечный скроллинг;
- роутинг.

2.4 Задание № 4. Аутентификация и авторизация

В заданиях № 1–3 реализуйте аутентификацию, регистрацию и авторизацию пользователей с использованием JWT.

Объекты должны отображаться, изменяться и удаляться только пользователем, который их создал.

2.5 Задание № 5. Микросервисная архитектура

Реализуйте сервис, полученный в заданиях № 1–4, в микросервисной архитектуре:

- отдельный микросервис для аутентификации и регистрации;
- отдельный микросервис для REST API предметной области;
- отдельный микросервис для хостинга React-приложения;
- отдельные контейнеры PostgreSQL для хранения данных.

2.6 Задание № 6. Автоматическое тестирование микросервисов

Реализуйте автоматическое тестирование разработанных микросервисов.

2.7 Задание № 7. Автоматическое UI-тестирование

Реализуйте автоматическое UI-тестирование клиентской части приложения.

2.8 Экзамен

Экзамен состоит в защите проекта (сервер + клиент), созданного в течение семестра.

В качестве альтернативы можно защитить свой проект, реализованный вне пар и для других целей, в случае, если в нем реализован REST API-сервис (на любом языке), веб-клиент с использованием реактивных фреймворков, а также автоматическое тестирование (в том числе UI).

3 Практические задания (весенний семестр)

3.1 Задание № 1. Kafka и обработка событий

Реализуйте запуск Kafka в отдельном контейнере.

При любых изменениях сущности (create/update/delete) публикуйте событие в Kafka-топик (например, `ExpenseCreated`, `OrderPlaced`).

Реализуйте простой consumer, который логирует события, а также consumer для записи в БД.

Обеспечьте идемпотентность обработки сообщений consumers Kafka.

3.2 Задание № 2. Event Sourcing

Реализуйте Event Sourcing для сущности (например, Expense).

Храните события в Kafka-топике.

Реализуйте replay для получения текущего состояния.

Реализуйте чистую реализацию Event Sourcing (в учебных целях), без какой-либо оптимизации чтения.

3.3 Задание № 3. Saga Choreography

Придумайте и реализуйте сложную бизнес-операцию в своем домене, требующую координации (например, «Оформление заказа с проверкой наличия», «Регистрация рейса с резервированием слотов», «Обработка обращения с уведомлением департамента»).

Реализуйте Saga Choreography.

3.4 Задание № 4. Outbox pattern, Schema Registry и DLQ

1. Реализуйте Outbox pattern для всех случаев, где осуществляется запись в базу данных, сопровождаемая записью в Kafka (кроме choreography).

2. Реализуйте Schema Registry для используемых при передаче информации через Kafka данных (там, где не используется Outbox pattern).

Замечание. Учебное задание не подразумевает строгого соответствия стандартным практикам, а ориентировано на знакомство с различными подходами. Поэтому требование реализации Schema Registry только там, где не используется Outbox pattern, позволяет уменьшить объем работы.

3. Реализуйте механизм DLQ: при возникновении неустраняемой ошибки обработки события (например, после трех неудачных попыток) событие должно перенаправляться в отдельный DLQ-топик для последующей ручной обработки.

3.5 Зачёт

Зачёт состоит в простом подсчете средней оценки по выполненным заданиям. При этом учитываются срок сдачи, качество защиты и полнота выполнения.

Вместо стандартных заданий можно защищать сторонние проекты с аналогичными архитектурными решениями.